

LATIHAN PRAKTIKUM

MATA KULIAH PEMROGRAMAN MOBILE

PERTEMUAN 02

Validasi, UI States, Navigasi & Data Passing pada Project Kelompok



**DEPARTEMEN SISTEM INFORMASI
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS
TAHUN 2026**

Pengantar

Latihan ini melanjutkan UI hasil Praktikum 1 yang sudah ada di repository kelompok. **Setiap praktikan mengerjakan semua langkah secara mandiri.** Setiap langkah mencantumkan bagian modul yang menjadi acuan.

Catatan: Aturan

- Kerjakan di branch latihan2-NIM. **Jangan push ke main.**
- Tampilan dari Praktikum 1 tidak diubah. Yang ditambahkan hanya logika.
- Sesuaikan nama class pada contoh (LoginScreen, HomeScreen, DetailScreen) dengan nama pada project masing-masing. Jika belum ada layar login, buat dari contoh di Langkah 4.
- Setelah praktikum, kelompok bebas memilih hasil anggota mana yang di-merge ke main.

Hasil akhir latihan:

```
Login --(isian valid)-----> Home
Home  --(tap item + data)-----> Detail
Detail --(Tulis Catatan)-----> Form Catatan
Form Catatan --(Simpan + teks)---> Detail (catatan tampil)
```

1. Clone repo dan buat branch pribadi
2. Buat model dan data dummy
3. Buat file pendukung
4. Login: form dan validasi
5. Home: status loading, kosong, dan error
6. Named routes: Login → Home
7. Home → Detail (mengirim data)
8. Detail → Form Catatan → Detail (menerima data)
9. Uji akhir, commit, dan push

Langkah 1 — Clone Repo dan Buat Branch Pribadi

Jalankan di terminal. Ganti alamat repository dan NIM sesuai milik masing-masing.

```
git clone https://github.com/USERNAME/NAMA-PROJECT-KELOMPOK.git
cd NAMA-PROJECT-KELOMPOK
git checkout -b latihan2-2311522xxx
flutter pub get
flutter run
```

Checkpoint: Hasil yang diharapkan

- Aplikasi hasil Praktikum 1 berjalan.
- Branch aktif adalah latihan2-NIM (cek dengan git branch).

Langkah 2 — Buat Model dan Data Dummy

Acuan: Modul Bagian 1.C.

Buat folder models dan data di dalam lib/, lalu buat dua file berikut. Nama class dan isi data boleh disesuaikan dengan aplikasi kelompok (misalnya Produk), asalkan konsisten.

lib/models/item.dart

```
/// Model data utama aplikasi.  
/// Boleh diganti nama class & field-nya sesuai aplikasi kelompok  
/// (misalnya Produk, Buku, Kegiatan, Lapangan).  
class Item {  
  final String id;  
  final String title;  
  final String subtitle;  
  final String description;  
  
  const Item({  
    required this.id,  
    required this.title,  
    required this.subtitle,  
    required this.description,  
  });  
}
```

lib/data/item_repository.dart

```
import '../models/item.dart';

/// Sumber data sementara (dummy). Nanti bisa diganti dengan API/database.
class ItemRepository {
  // Ganti isi data dummy ini sesuai aplikasi kelompok
  static const List<Item> _items = [
    Item(
      id: '1',
      title: 'Judul item 1',
      subtitle: 'Keterangan singkat item 1',
      description: 'Deskripsi lengkap item 1.',
    ),
    Item(
      id: '2',
      title: 'Judul item 2',
      subtitle: 'Keterangan singkat item 2',
      description: 'Deskripsi lengkap item 2.',
    ),
    Item(
      id: '3',
      title: 'Judul item 3',
      subtitle: 'Keterangan singkat item 3',
      description: 'Deskripsi lengkap item 3.',
    ),
  ];

  /// Mengambil daftar item.
  /// simulateError: true -> sengaja dibuat gagal untuk menguji error state.
  Future<List<Item>> fetchItems({bool simulateError = false}) async {
    await Future.delayed(const Duration(seconds: 2)); // simulasi waktu tunggu server
    if (simulateError) {
      throw Exception('Gagal memuat data. Periksa koneksi internet.');
```

Langkah 3 — Buat File Pendukung

Acuan: Modul Bagian 2.E, Bagian 3.C, Bagian 4.C, dan Lampiran A.

Buat tiga file berikut dan salin isinya tanpa diubah.

lib/utils/validators.dart

```
/// Kumpulan fungsi validasi yang bisa dipakai ulang (reusable).
/// Aturan validator di Flutter:
///   - return null      -> input VALID
///   - return 'pesan'   -> input TIDAK VALID, pesan tampil di bawah field
class Validators {
  Validators._();

  static String? requiredField(String? value, {String fieldName = 'Field ini'}) {
    if (value == null || value.trim().isEmpty) {
      return '$fieldName wajib diisi';
    }
    return null;
  }
}
```

```
static String? email(String? value) {
  final requiredError = requiredField(value, fieldName: 'Email');
  if (requiredError != null) return requiredError;

  final emailRegex = RegExp(r'^[\w.+-]+@[ \w-]+(\.[ \w-]+)?$');
  if (!emailRegex.hasMatch(value!.trim())) {
    return 'Format email tidak valid';
  }
  return null;
}

static String? password(String? value) {
  if (value == null || value.isEmpty) return 'Password wajib diisi';
  if (value.length < 8) return 'Password minimal 8 karakter';
  return null;
}

static String? minLength(
  String? value,
  int min, {
  String fieldName = 'Field ini',
}) {
  final requiredError = requiredField(value, fieldName: fieldName);
  if (requiredError != null) return requiredError;
  if (value!.trim().length < min) {
    return '$fieldName minimal $min karakter';
  }
  return null;
}

static String? rating(String? value) {
  final requiredError = requiredField(value, fieldName: 'Rating');
  if (requiredError != null) return requiredError;

  final number = int.tryParse(value!.trim());
  if (number == null) return 'Rating harus berupa angka';
  if (number < 1 || number > 5) return 'Rating harus di antara 1 sampai 5';
  return null;
}
}
```

lib/widgets/state_views.dart

```
import 'package:flutter/material.dart';

/// Tampilan saat data sedang dimuat.
class LoadingView extends StatelessWidget {
  final String message;

  const LoadingView({super.key, this.message = 'Memuat data...'});

  @override
  Widget build(BuildContext context) {
    return Center(
      child: Column(
        mainAxisAlignment: MainAxisAlignment.min,
        children: [
          const CircularProgressIndicator(),
          const SizedBox(height: 16),
          Text(message),
        ],
      ),
    );
  }
}
```

```

    ),
  );
}
}

/// Tampilan saat data berhasil dimuat tetapi kosong.
class EmptyView extends StatelessWidget {
  final String message;
  final IconData icon;

  const EmptyView({
    super.key,
    required this.message,
    this.icon = Icons.inbox_outlined,
  });

  @override
  Widget build(BuildContext context) {
    return Center(
      child: Padding(
        padding: const EdgeInsets.all(24),
        child: Column(
          mainAxisAlignment: MainAxisAlignment.min,
          children: [
            Icon(icon, size: 64, color: Theme.of(context).colorScheme.outline),
            const SizedBox(height: 12),
            Text(message, textAlign: TextAlign.center),
          ],
        ),
      ),
    );
  }
}

/// Tampilan saat terjadi error, lengkap dengan tombol "Coba Lagi".
class ErrorView extends StatelessWidget {
  final String message;
  final VoidCallback onRetry;

  const ErrorView({super.key, required this.message, required this.onRetry});

  @override
  Widget build(BuildContext context) {
    final colorScheme = Theme.of(context).colorScheme;
    return Center(
      child: Padding(
        padding: const EdgeInsets.all(24),
        child: Column(
          mainAxisAlignment: MainAxisAlignment.min,
          children: [
            Icon(Icons.cloud_off, size: 64, color: colorScheme.error),
            const SizedBox(height: 12),
            Text(
              'Oops, terjadi kesalahan',
              style: Theme.of(context).textTheme.titleMedium,
            ),
            const SizedBox(height: 8),
            Text(message, textAlign: TextAlign.center),
            const SizedBox(height: 16),
            FilledButton.icon(
              onPressed: onRetry,
              icon: const Icon(Icons.refresh),
              label: const Text('Coba Lagi'),
            ),
          ],
        ),
      ),
    );
  }
}

```

```

    ),
  ],
),
),
);
}
}

/// Banner pesan error kecil (dipakai di dalam form).
class ErrorBanner extends StatelessWidget {
  final String message;

  const ErrorBanner({super.key, required this.message});

  @override
  Widget build(BuildContext context) {
    final colorScheme = Theme.of(context).colorScheme;
    return Container(
      padding: const EdgeInsets.all(12),
      decoration: BoxDecoration(
        color: colorScheme.errorContainer,
        borderRadius: BorderRadius.circular(8),
      ),
      child: Row(
        children: [
          Icon(Icons.error_outline, color: colorScheme.onErrorContainer),
          const SizedBox(width: 8),
          Expanded(
            child: Text(
              message,
              style: TextStyle(color: colorScheme.onErrorContainer),
            ),
          ),
        ],
      ),
    );
  }
}

```

lib/screens/not_found_screen.dart

```

import 'package:flutter/material.dart';

import '../routes/app_routes.dart';

/// Halaman cadangan ketika route tidak dikenal atau argument tidak valid.
class NotFoundScreen extends StatelessWidget {
  final String? routeName;
  final String message;

  const NotFoundScreen({
    super.key,
    this.routeName,
    this.message = 'Halaman yang kamu tuju belum tersedia.',
  });

  @override
  Widget build(BuildContext context) {
    final textTheme = Theme.of(context).textTheme;
    return Scaffold(
      appBar: AppBar(title: const Text('Halaman Tidak Ditemukan')),
      body: Center(

```

```

child: Padding(
  padding: const EdgeInsets.all(24),
  child: Column(
    mainAxisAlignment: MainAxisAlignment.min,
    children: [
      const Icon(Icons.search_off, size: 72),
      const SizedBox(height: 8),
      Text('404', style: textTheme.displaySmall),
      const SizedBox(height: 8),
      Text(message, textAlign: TextAlign.center),
      if (routeName != null) ...[
        const SizedBox(height: 4),
        Text('Route: $routeName', style: textTheme.bodySmall),
      ],
      const SizedBox(height: 16),
      FilledButton(
        onPressed: () {
          if (Navigator.canPop(context)) {
            Navigator.pop(context);
          } else {
            Navigator.pushReplacementNamed(context, AppRoutes.login);
          }
        },
        child: const Text('Kembali'),
      ),
    ],
  ),
),
);
}

```

Catatan

Error pada baris import `'../routes/app_routes.dart';` di `not_found_screen.dart` akan hilang setelah Langkah 6.

Langkah 4 — Login: Form dan Validasi

Acuan: Modul Bagian 2.B–2.E.

Buka file layar login. Nomor (1)–(6) sesuai komentar pada contoh kode.

1. Ubah menjadi `StatefulWidget`: klik nama class → `Ctrl + .` → *Convert to StatefulWidget*. Import `validators.dart`.
2. Tambahkan **(1)** `_formKey` dan controller, **(2)** `dispose()`, dan **(3)** `_submit()`.
3. **(4)** Bungkus `Column` berisi field dengan `Form` (`Ctrl + .` → *Wrap with widget*).
4. **(5)** Ganti `TextField` menjadi `TextFormField`, tambahkan controller dan validator.
5. **(6)** Isi tombol login dengan `onPressed: _submit`.

Karena route `Home` baru dibuat di Langkah 6, isi dulu `_submit()` seperti ini:

```

void _submit() {
  final isValid = _formKey.currentState?.validate() ?? false;
  if (!isValid) return;
  ScaffoldMessenger.of(context).showSnackBar(
    const SnackBar(content: Text('Form valid!')),
  );
}

```

Contoh hasil akhir (setelah Langkah 6):

lib/screens/login_screen.dart

```

import 'package:flutter/material.dart';

import '../routes/app_routes.dart';
import '../utils/validators.dart';

class LoginScreen extends StatefulWidget {
  const LoginScreen({super.key});

  @override
  State<LoginScreen> createState() => _LoginScreenState();
}

class _LoginScreenState extends State<LoginScreen> {
  // (1) key untuk Form dan controller untuk setiap field
  final _formKey = GlobalKey<FormState>();
  final _emailController = TextEditingController();
  final _passwordController = TextEditingController();

  // (2) buang controller saat layar ditutup
  @override
  void dispose() {
    _emailController.dispose();
    _passwordController.dispose();
    super.dispose();
  }

  // (3) dijalankan saat tombol Masuk ditekan
  void _submit() {
    final isValid = _formKey.currentState?.validate() ?? false;
    if (!isValid) return; // ada isian yang salah -> berhenti

    // Isian benar -> pindah ke Home, Login dibuang dari stack
    Navigator.pushReplacementNamed(context, AppRoutes.home);
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      body: SafeArea(
        child: Center(
          child: SingleChildScrollView(
            padding: const EdgeInsets.all(24),
            // (4) bungkus field dengan Form
            child: Form(
              key: _formKey,
              autovalidateMode: AutovalidateMode.onUserInteraction,
              child: Column(
                crossAxisAlignment: CrossAxisAlignment.stretch,
                children: [

```

```

Text(
  'Masuk',
  textAlign: TextAlign.center,
  style: Theme.of(context).textTheme.headlineMedium,
),
const SizedBox(height: 32),
// (5) TextField diganti TextFormField + validator
TextFormField(
  controller: _emailController,
  keyboardType: TextInputType.emailAddress,
  decoration: const InputDecoration(
    labelText: 'Email',
    border: OutlineInputBorder(),
  ),
  validator: Validators.email,
),
const SizedBox(height: 16),
TextFormField(
  controller: _passwordController,
  obscureText: true,
  decoration: const InputDecoration(
    labelText: 'Password',
    border: OutlineInputBorder(),
  ),
  validator: Validators.password,
),
const SizedBox(height: 24),
// (6) tombol memanggil _submit
FilledButton(
  onPressed: _submit,
  child: const Text('Masuk'),
),
),
),
),
),
),
),
);
}
}

```

Jika field login bukan email, gunakan misalnya validator: (value) => Validators.minLength(value, 3, fieldName: 'Username').

No	Yang dilakukan	Hasil yang benar
1	Tekan login tanpa mengisi apa pun	Muncul "... wajib diisi" di setiap field
2	Isi email abc	Muncul "Format email tidak valid"
3	Isi password 123	Muncul "Password minimal 8 karakter"
4	Isi semua dengan benar, tekan login	Muncul SnackBar "Form valid!"

Langkah 5 — Home: Status Loading, Kosong, dan Error

Acuan: Modul Bagian 3.A–3.D.

Buka file layar Home.

1. Ubah menjadi StatefulWidget. Import `item_repository.dart`, `item.dart`, dan `state_views.dart`.
2. Tambahkan **(1)** enum `ViewStatus`, **(2)** variabel `state`, **(3)** `initState()`, dan **(4)** `_loadItems()`.
3. **(5)** Ganti daftar yang masih hardcode di body dengan `_buildContent()`.
4. **(6)** Tambahkan `_buildContent()` dan `_buildList()`. Widget kartu dari Praktikum 1 boleh dipakai menggantikan `ListTile`.
5. Isi dulu `onTap: () {}`. Bagian **(7)** diisi di Langkah 7.

Contoh hasil akhir (setelah Langkah 7):

lib/screens/home_screen.dart

```
import 'package:flutter/material.dart';

import '../data/item_repository.dart';
import '../models/item.dart';
import '../routes/app_routes.dart';
import '../widgets/state_views.dart';

// (1) status tampilan
enum ViewStatus { loading, success, error }

class HomeScreen extends StatefulWidget {
  const HomeScreen({super.key});

  @override
  State<HomeScreen> createState() => _HomeScreenState();
}

class _HomeScreenState extends State<HomeScreen> {
  // (2) variabel state
  final _repository = ItemRepository();
  ViewStatus _status = ViewStatus.loading;
  List<Item> _items = [];
  String _errorMessage = '';
  bool _simulateError = false; // ubah ke true untuk menguji error state

  // (3) ambil data saat layar pertama kali dibuka
  @override
  void initState() {
    super.initState();
    _loadItems();
  }

  // (4) mengambil data + menangani error
  Future<void> _loadItems() async {
    if (_status != ViewStatus.loading) {
      setState(() => _status = ViewStatus.loading);
    }
    try {
      final items = await _repository.fetchItems(simulateError: _simulateError);
      if (!mounted) return;
      setState(() {
        _items = items;
      });
    } catch (e) {
      _errorMessage = e.toString();
      _status = ViewStatus.error;
    }
  }

  // (5) build content
  Widget _buildContent() {
    if (_status == ViewStatus.loading) {
      return Center(child: CircularProgressIndicator());
    } else if (_status == ViewStatus.error) {
      return Center(child: Text(_errorMessage));
    } else {
      return ListView.builder(
        itemCount: _items.length,
        itemBuilder: (context, index) {
          return Card(
            child: ListTile(
              title: Text(_items[index].title),
              subtitle: Text(_items[index].description),
              trailing: Text(_items[index].price.toString()),
            ),
          );
        },
      );
    }
  }

  // (6) build list
  Widget _buildList() {
    return _buildContent();
  }

  // (7) onTap
  void _onTap() {}
}
```

```

        _status = ViewStatus.success;
    });
} catch (e) {
    if (!mounted) return;
    setState(() {
        _errorMessage = e.toString().replaceFirst('Exception: ', '');
        _status = ViewStatus.error;
    });
}
}

@override
Widget build(BuildContext context) {
    return Scaffold(
        appBar: AppBar(title: const Text('Home')),
        body: _buildContent(), // (5) isi layar tergantung status
    );
}

// (6) memilih tampilan: loading / error / empty / daftar data
Widget _buildContent() {
    return switch (_status) {
        ViewStatus.loading => const LoadingView(),
        ViewStatus.error => ErrorView(message: _errorMessage, onRetry: _loadItems),
        ViewStatus.success => _buildList(),
    };
}

Widget _buildList() {
    if (_items.isEmpty) {
        return const EmptyView(message: 'Belum ada data.');
```

Untuk menguji sebelum Langkah 6, ganti sementara home: di main.dart menjadi const HomeScreen(), lalu hot restart.

No	Yang dilakukan	Hasil yang benar
1	Buka Home	Loading ±2 detik, lalu daftar item tampil

No	Yang dilakukan	Hasil yang benar
2	Ubah <code>_simulateError = true</code> , hot restart	Muncul pesan error dan tombol "Coba Lagi"
3	Kembalikan ke <code>false</code> , tekan "Coba Lagi"	Loading, lalu daftar item tampil
4	Kosongkan sementara <code>list_items</code> di repository	Muncul "Belum ada data."

Langkah 6 — Named Routes: Login ke Home

Acuan: Modul Bagian 4.B–4.D.

1. Buat `lib/routes/app_routes.dart` seperti kode di bawah.
2. Di `main.dart`, hapus `home: ...` dan tambahkan `initialRoute`, `onGenerateRoute`, dan `onUnknownRoute`. Import `routes/app_routes.dart`.
3. Di `_submit()` login, ganti `SnackBar` dengan `Navigator.pushReplacementNamed(context, AppRoutes.home);` dan import `app_routes.dart`.
4. Hot restart (R).

lib/routes/app_routes.dart

```
import 'package:flutter/material.dart';

import '../screens/home_screen.dart';
import '../screens/login_screen.dart';
import '../screens/not_found_screen.dart';

class AppRoutes {
  AppRoutes._();

  static const String login = '/login';
  static const String home = '/home';

  static Route<dynamic>? onGenerateRoute(RouteSettings settings) {
    switch (settings.name) {
      case login:
        return MaterialPageRoute<void>(
          builder: (_) => const LoginScreen(),
          settings: settings,
        );
      case home:
        return MaterialPageRoute<void>(
          builder: (_) => const HomeScreen(),
          settings: settings,
        );
      default:
        return null; // route tidak terdaftar -> halaman 404
    }
  }

  static Route<dynamic> onUnknownRoute(RouteSettings settings) {
    return MaterialPageRoute<void>(
      builder: (_) => NotFoundScreen(routeName: settings.name),
      settings: settings,
    );
  }
}
```

lib/main.dart (bagian MaterialApp)

```
return MaterialApp(
  theme: ..., // tetap dari Praktikum 1
  initialRoute: AppRoutes.login,
  onGenerateRoute: AppRoutes.onGenerateRoute,
  onUnknownRoute: AppRoutes.onUnknownRoute,
);
```

Checkpoint: Hasil yang diharapkan

- Aplikasi dimulai dari Login; login benar pindah ke Home.
- Tombol back di Home tidak kembali ke Login.

Langkah 7 — Home ke Detail (Mengirim Data)

Acuan: Modul Bagian 4.E.

1. Di layar Detail, import `item.dart`, tambahkan `final Item item;` dan constructor `const DetailScreen({super.key, required this.item});`. Ganti teks yang masih `hardcode` dengan data `item` (misalnya `Text(item.title)`).
2. Di `app_routes.dart`, tambahkan konstanta dan case `detail` di bawah, serta import `item.dart` dan `detail_screen.dart`.
3. Di Home, isi `onTap (7)` dengan `Navigator.pushNamed(context, AppRoutes.detail, arguments: item)` dan import `app_routes.dart`.
4. Hot restart.

Tambahan di `app_routes.dart`

```
static const String detail = '/detail';

// di dalam switch, sebelum default
case detail:
  final args = settings.arguments;
  if (args is Item) {
    return MaterialPageRoute<void>(
      builder: (_) => DetailScreen(item: args),
      settings: settings,
    );
  }
  return null; // data salah/kosong -> halaman 404
```

Checkpoint: Hasil yang diharapkan

- Item berbeda membuka Detail dengan data yang berbeda.
- Back dari Detail kembali ke Home.

Langkah 8 — Detail ke Form Catatan (Menerima Data)

Acuan: Modul Bagian 4.F.

1. Buat `lib/screens/catatan_form_screen.dart` dari kode di bawah.
2. Di `app_routes.dart`, tambahkan konstanta dan case `catatanForm` di bawah, serta import `catatan_form_screen.dart`.
3. Ubah Detail menjadi `StatefulWidget`. Data `item` sekarang dibaca dengan `widget.item`.
4. Di Detail, tambahkan **(2)** `_catatan`, **(3)** `_bukaFormCatatan()`, teks catatan, dan tombol "Tulis Catatan" (lihat contoh).
5. Hot restart.

`lib/screens/catatan_form_screen.dart`

```
import 'package:flutter/material.dart';
```

```
import '../utils/validators.dart';

class CatatanFormScreen extends StatefulWidget {
  const CatatanFormScreen({super.key});

  @override
  State<CatatanFormScreen> createState() => _CatatanFormScreenState();
}

class _CatatanFormScreenState extends State<CatatanFormScreen> {
  final _formKey = GlobalKey<FormState>();
  final _catatanController = TextEditingController();

  @override
  void dispose() {
    _catatanController.dispose();
    super.dispose();
  }

  void _simpan() {
    final isValid = _formKey.currentState?.validate() ?? false;
    if (!isValid) return;

    // Tutup layar ini sambil MEMBAWA teks catatan ke layar sebelumnya
    Navigator.pop(context, _catatanController.text.trim());
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Tulis Catatan')),
      body: SingleChildScrollView(
        padding: const EdgeInsets.all(16),
        child: Form(
          key: _formKey,
          autovalidateMode: AutovalidateMode.onUserInteraction,
          child: Column(
            crossAxisAlignment: CrossAxisAlignment.stretch,
            children: [
              TextFormField(
                controller: _catatanController,
                maxLines: 3,
                decoration: const InputDecoration(
                  labelText: 'Catatan',
                  border: OutlineInputBorder(),
                ),
                validator: (value) =>
                  Validators.minLength(value, 5, fieldName: 'Catatan'),
              ),
              const SizedBox(height: 16),
              FilledButton(
                onPressed: _simpan,
                child: const Text('Simpan'),
              ),
            ],
          ),
        ),
      ),
    );
  }
}
```

Tambahan di app_routes.dart

```
static const String catatanForm = '/catatan-form';

// di dalam switch, sebelum default
case catatanForm:
  // <String> karena layar ini mengembalikan teks saat ditutup
  return MaterialPageRoute<String>(
    builder: (_) => const CatatanFormScreen(),
    settings: settings,
  );
```

Contoh: lib/screens/detail_screen.dart

```
import 'package:flutter/material.dart';

import '../models/item.dart';
import '../routes/app_routes.dart';

class DetailScreen extends StatefulWidget {
  // (1) data yang DITERIMA dari Home
  final Item item;

  const DetailScreen({super.key, required this.item});

  @override
  State<DetailScreen> createState() => _DetailScreenState();
}

class _DetailScreenState extends State<DetailScreen> {
  // (2) menyimpan catatan yang dikirim balik dari form
  String? _catatan;

  // (3) buka form, TUNGGU hasilnya, lalu tampilkan
  Future<void> _bukaFormCatatan() async {
    final hasil = await Navigator.pushNamed<String>(
      context,
      AppRoutes.catatanForm,
    );
    if (!mounted || hasil == null) return; // null = pengguna batal

    setState(() => _catatan = hasil);
    ScaffoldMessenger.of(context).showSnackBar(
      const SnackBar(content: Text('Catatan berhasil disimpan')),
    );
  }

  @override
  Widget build(BuildContext context) {
    // Di dalam State, data widget dibaca dengan "widget.item"
    final item = widget.item;

    return Scaffold(
      appBar: AppBar(title: Text(item.title)),
      body: ListView(
        padding: const EdgeInsets.all(16),
        children: [
          Text(item.title, style: Theme.of(context).textTheme.headlineSmall),
          const SizedBox(height: 4),
          Text(item.subtitle),
          const SizedBox(height: 16),
        ],
      ),
    );
  }
}
```

```

        Text(item.description),
        const Divider(height: 32),
        Text(
          _catatan == null ? 'Belum ada catatan.' : 'Catatan: $_catatan',
        ),
        const SizedBox(height: 16),
        FilledButton.icon(
          onPressed: _bukaFormCatatan,
          icon: const Icon(Icons.edit_note),
          label: const Text('Tulis Catatan'),
        ),
      ],
    ),
  );
}
}

```

No	Yang dilakukan	Hasil yang benar
1	Tekan Simpan tanpa mengisi	Muncul "Catatan wajib diisi"
2	Isi abc, tekan Simpan	Muncul "Catatan minimal 5 karakter"
3	Isi dengan benar, tekan Simpan	Kembali ke Detail, catatan tampil, muncul Snackbar
4	Buka form lagi, tekan back	Catatan lama tidak berubah

Langkah 9 — Uji Akhir, Commit, dan Push

Hot restart, lalu pastikan semua pengujian berikut berhasil.

No	Pengujian	Berhasil
1	Login dengan isian salah menampilkan pesan error	☐
2	Login benar pindah ke Home; back tidak kembali ke Login	☐
3	Home menampilkan loading, lalu daftar data	☐
4	Dengan <code>_simulateError = true</code> , Home menampilkan error + "Coba Lagi"	☐
5	Tap item membuka Detail dengan data yang sesuai	☐
6	Catatan dari form tampil di Detail	☐
7	Route yang tidak terdaftar menampilkan halaman 404	☐

Untuk pengujian 7, tambahkan sementara tombol dengan `onPressed: () => Navigator.pushNamed(context, '/tidak-ada')`, lalu hapus setelah diuji.

```
git add .  
git commit -m "feat: latihan modul 2"  
git push -u origin latihan2-2311522xxx
```

Jika Menggunakan AI

Boleh, asalkan kamu bisa menjelaskan setiap baris kodenya. Contoh prompt:

```
Saya mengerjakan Langkah [nomor] latihan Modul 2 Flutter.  
Kode layar saya saat ini:  
[tempel kode]  
  
Tambahkan: [salin instruksi langkah tersebut].  
Jangan ubah tampilan. Gunakan Form + TextFormField + validator dari  
lib/utils/validators.dart, navigasi dengan AppRoutes (bukan Get.to),  
dan jelaskan setiap perubahan.
```